Probeklausur Informatik I
Wintersemester 2008/09
Prof. Dr. Carsten Damm
20. Januar 2009
von 16:00+x bis 17:30+x Uhr
(Bearbeitungszeit: 90 Minuten)

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc..

Bleistift ist nicht erlaubt. Korrekturfelder | | **nicht beschriften**!

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Auswahlaufgabe)	5+0	
Aufgabe 2 (Java: eine einfache Klasse)	11+0	
Aufgabe 3 (Datenstrukturen und Graphen)	13+0	
Aufgabe 4 (Zahlendarstellung)	14+0	
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	16+2	
Aufgabe 6 (Sortieren und Schleifeninvariante)	15+0	
Aufgabe 7 (Rekursion und Iteration)	16+2	
Summe	90+4	

Note:

Name: MatNr.:

Aufgabe 1 (Auswahlaufgabe [5+0 Punkte])

Nur jeweils eine Antwort ist richtig. Falsche Antworten werden innerhalb dieser Aufgabe verrechnet (Bsp.: a) und b) richtig, aber c), d) und e) falsch ergibt 0P).

Kreisen Sie die richtige Nummer ein. Mehr Kreise (oder unleserlich) ergeben 0P für die Teilaufgabe.

a) Überladen einer Methode

1. ist die Zuordnung mehrerer Semantiken zum gleichen Methodennamen (Auswahl erfolgt durch Aufrufkontext)
2. ist die Programmierung unübersichtlich vieler verschiedener Funktionalitäten innerhalb einer Methode
3. geschieht zum Beispiel bei der Neudefinition der Methode `Object.toString()` in einer abgeleiteten Klasse

b) imperative Programmierung ist gekennzeichnet durch

1. Kommandozeilensteuerung
2. Befehlsfolgen zur Zustandsänderung
3. Zerlegen von Problemen in Teilprobleme

c) Rekursion ist

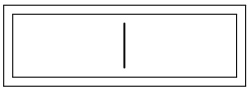
1. das Rückverfolgen von Methodenaufrufen
2. das Aufrufen einer Methode in deren eigenem Methodenrumpf
3. das Rückführen einer Problemlösung auf zwei kleinere Probleme

d) Aus welchen drei Kontrollstrukturen lassen sich alle Algorithmen aufbauen?

1. Eingabe, Verarbeitung und Ausgabe
2. for-Schleife, while-Schleife und do-while-Schleife
3. Komposition, Alternative und while-Schleife

e) Algorithmen sind stets

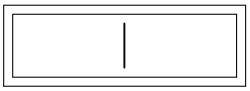
1. in endlicher Zeit abarbeitbar
2. endlich beschreibbar
3. in asymptotischer Form analysierbar



Name: MatNr.:

Aufgabe 2 (Java: eine einfache Klasse [11+0 Punkte])
Geben Sie eine einfache Klasse `Modul` an, die man zur Verwaltung der Module in einem Bachelorstudiengang benutzen kann. Jedes Modul zeichnet sich durch seinen Namen, z.B. “Info I” oder “C-Kurs”, sowie seine Leistungspunktezahl aus. Außerdem will man jederzeit die Anzahl angebotener Module ermitteln können, sowie die Gesamtanzahl der Leistungspunkte in diesen. *Benutzen Sie in Ihrem Quelltext die Bezeichner* `name`, `punkte`, `anzahl` *und* `gesamtPunkte` *für diese Dinge.* Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und der Leistungspunktezahl,
- Setzen und Abfragen der Punktzahl jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele Leistungspunkte diese zusammen haben.



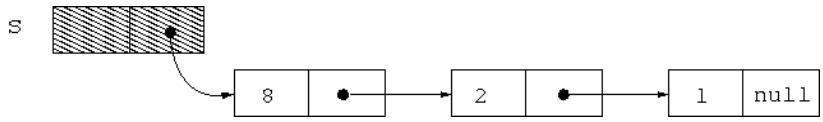
Name: MatNr.:

Aufgabe 3 (Datenstrukturen und Graphen [13+0 Punkte])

a) (3P) Auf einen anfangs leeren Stack werden `push/pop`-Operationen angewendet. Tragen Sie den Zustand des Stacks nach den angegebenen Aufrufen ein (erste Zeile ist komplett vorgegeben). Bitte schreiben Sie deutlich!

`push(1);push(2);push(8);`	1	2	8									
`push(5);push(3);pop();`												
`push(1);pop();pop();`												
`pop();push(7);push(1);`												

b) (6P)

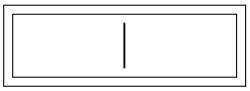


Dies skizziert die Realisierung eines Stacks durch eine verkettete Liste mit Startknoten `s` (Zustand wie erste Tabellenzeile oben). Mögliche Befehle:

- *n* `= neu(`*i*`)` — Erzeugen von neuem Knoten namens *n* mit Inhalt *i*
- *x* `= inhalt(`*m*`)` — kopiere in *m* gespeicherte Zahl nach *x*
- `nachfolger(`*o*`) = `*p* — der Nachfolger von *o* ist nun *p*
  (oder auch: *k* `= nachfolger(`*z*`)` — Nachfolger von *z* in *k* speichern)

Geben Sie je eine Befehlsfolge an, um (1) bzw. (2) zu realisieren:
(1) `x = top(); pop();` (2) `push(5);`
Auf Reihenfolge achten! Auf Variablendeklarationen, Leerheitstest o.ä. brauchen Sie nicht achten!

c) (4P) Welche Tiefe hat ein binärer Baum mit n Blättern mindestens? Geben Sie den Wert möglichst genau an und begründen Sie Ihre Antwort.



Name: MatNr.:

Aufgabe 4 (Zahlendarstellung [14+0 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen bekanntlich mit dem Zweierkomplement der Länge 4 dar (4-Bit-Zweierkomplement). Ergänzen Sie die folgenden Konstantendefinitionen der Wrapper-Klasse `Ginteger`:

```
public static final gint MAX_VALUE = ........;

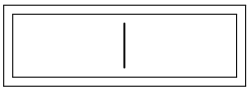
public static final gint MIN_VALUE = ........;
```

b) (5P) Wir betrachten dazu noch den Datentyp `glint`: ganze Zahlen werden hier mit der 8-Bit-Zweierkomplement dargestellt. Ermitteln Sie die Bitmuster für 40 und -81.

c) (6P) Ergänzen Sie diese Tabelle von `glint`-Werten.

Bitte deutlich schreiben, Nebenrechnung oder Probe nur auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13			
	$(10001011)_{\overline{2}}$		
		$(357)_8$	
			$(1E)_{16}$



Name: MatNr.:

Aufgabe 5 (Algorithmen-Entwurf und Analyse [16+2 Punkte])
Gegeben ist eine nur aus "+" oder "-" bestehende endliche Zeichenfolge, z.B. "`--+++--+++++-+-+-+-+--+`"
gespeichert in einem Feld `z[0]...z[k-1]`.

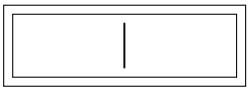
a) (8P + (2 Zusatzpunkte für (asymptotisch) optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus "+"-Zeichen besteht (im Beispiel: 5).

Der Algorithmus soll entweder in Java- oder Pseudocode- oder als Programmablaufplan oder als klar formulierter Text beschrieben werden. |

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung). |

c) (3P) Ein Algorithmus zur Lösung von a) liest beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht. Der Algorithmus kann die Feldlänge im vorhinein ermitteln.

Geben Sie Zeichenketten (1) und (2) der Länge 8 wie folgt an: (1) Es muss bis zum Ende gelesen werden. (2) Es muss *nicht* bis zum Ende gelesen werden und das Resultat ist 4. Begründen Sie jeweils kurz. |



Name: MatNr.:

Aufgabe 6 (Sortieren und Schleifeninvariante [15+0 Punkte])
Wir betrachten folgenden elementaren Sortieralgorithmus

```
30    public static void selectionSort(int[] feld) {
31        int maxpos, tmp;
32        for (int i=feld.length; i>1; i--) {
33            // bestimme Position des Maximums im Intervall [0,...,i-1]
34            maxpos = 0; // Position des Maximums
35            for (int j=0; j<i; j++)
36                if ( feld[j] > feld[maxpos])
37                    maxpos = j;
38            // tausche feld[i-1] und feld[maxpos]:
39            tmp = feld[i-1]; feld[i-1] = feld[maxpos]; feld[maxpos] = tmp;
40        }
41    }
```

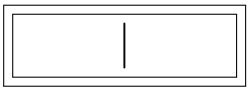
a) (4P) Wenden Sie den Algorithmus an auf
`int[] test = {1, 6, 9, 8, 2, 3, 4, 5, 7};`
und ergänzen Sie die Tabelle um die Feldinhalte *nach* Iterationen 1, 2 und 3 der *äußeren* `for`*-Schleife* (Zeilen 32-40).

Start	1	6	9	8	2	3	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

b) (5P) Geben Sie (mit Begründung) eine möglichst gute asymptotische Abschätzung der Laufzeit des Algorithmus auf Eingaben der Länge n an

c) (6P) Es gilt folgende Aussage: Nach Ausführung der inneren Schleife (Zeilen 35–37) gilt `feld[maxpos]` enthält das Maximum der Einträge im Teilfeld `feld[0..i-1]`.

Begründen Sie damit, dass das Feld nach `feld.length` − 1 Iterationen der *äußeren Schleife* (Zeilen 32–40) sortiert ist. Das formale Konzept der Schleifeninvariante können, müssen Sie aber nicht benutzen.



Name: MatNr.:

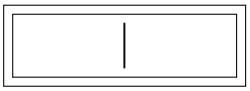
Aufgabe 7 (Rekursion und Iteration [16+2 Punkte])
Pseudocode ist ausreichend für diese Aufgaben. Die Zahlenfolge $a_0, a_1, a_2, \ldots$ ist definiert durch: $a_0 = 0, a_1 = 1$ und $a_n = -2a_{n-1} + 3a_{n-2}$ für $n > 2$.

a) (6P) Formulieren Sie einen *rekursiven* Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!). Zeichnen Sie den Aufrufbaum (Rekursionsbaum) bei Aufruf mit der Eingabe $n = 5$.

b) (6P) Formulieren Sie einen *iterativen*[2] Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!).

c) (4P + 2 Zusatzpunkte für Beweis der Formel) Für alle n gilt $a_n = \frac{1-(-3)^n}{4}$. Formulieren Sie aufgrund dessen einen Algorithmus, der bei Eingabe n (natürliche Zahl), den Wert a_n zurückgibt (nicht druckt!). Potenzieren darf dabei *nicht* benutzt werden.

[2]Zwei Möglichkeiten: (1) Feld anlegen, das nach und nach mit den Werten a_0, a_1, a_2, ... gefüllt wird oder (2) lediglich die letzten Werte speichern, daraus den nächsten berechnen und geeignet umspeichern usw.



Name: MatNr.: